τοποθετήσει τον σηματοφόρο (κόμμα). Με αυτόν τον σηματοφόρο ζητάμε ο αριθμός να εμφανιστεί με διαχωριστικά χιλιάδων, οπότε η έξοδος εδώ θα είναι 1.000.000. Επειδή όμως τα διαχωριστικά χιλιάδων δεν είναι ίδια για όλες τις χώρες, μας δίνεται η δυνατότητα να χρησιμοποιήσουμε μια άλλη έκδοση της printf που μας επιτρέπει να καθορίσουμε ποιο ακριβώς διαχωριστικό θα χρησιμοποιήσουμε. Έτσι, η γραμμή 4 εμφανίζει 1,000,000.

```
1     static void realF() { //Κώδικας 4.18
2       System.out.printf("%f%n", 5.1473);
3       System.out.printf("%(010.2f%n", -5.1473);
4       System.out.printf("%-10.2f%n", 5.1473);
5   }
```

***Κώδικας 4.18**: Διαμόρφωση εξόδου πραγματικών τύπων*

Η έξοδος του κώδικα 4.18 έχει ως εξής:

```
5,147300
(00005,15)
5,15
```

Στη γραμμή 3, ο σηματοφόρος '(' (αριστερή παρένθεση) έχει ως αποτέλεσμα να εμφανίζονται οι αρνητικοί αριθμοί εντός παρενθέσεων. Το δε 0 που ακολουθεί καλύπτει το πλάτος με μηδενικά στα αριστερά. Ο σηματοφόρος 0 δεν συντάσσεται με στοίχιση στα αριστερά. Κατά τα λοιπά, με όσα έχουμε συζητήσει μέχρι εδώ, είναι εύκολο να ερμηνεύσετε την έξοδο του κώδικα 4.18.

4.2.2 Η DecimalFormat

Ένας εναλλακτικός τρόπος για διαμόρφωση δεκαδικών αριθμών για έξοδο βασίζεται στην κλάση DecimalFormat. Η DecimalFormat παρέχει ευκολίες στη διαμόρφωση των αριθμών για έξοδο, εισάγει ωστόσο κάποιον βαθμό πολυπλοκότητας στον κώδικά μας.

```
1     static void outputFormat() { //Κώδικας 4.19
2         DecimalFormat f = new DecimalFormat("###,###.##");
3         double d1 = 9.566, d2 = 9.273;
4         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
5
6         d1 = 9.565;
7         d2 = 9.275;
8         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
9
10         d1 = 0.275;
11         d2 = 0.385;
12         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
13
14         d1 = 9_856_327.563;
15         d2 = 5627869.289;
16         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
17
18         f.applyPattern("###,###.###");
19         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
20
```

```
21          f.applyPattern("€###,###.###");
22          System.out.println(f.format(d1));

23          f.applyPattern("(€###,###.###)");
24          System.out.println(f.format(d1));
25          int k = 128;
26          System.out.println(f.format(k));
27
28          char c = 'A';
29          System.out.println(f.format(c));
30
31          f.applyPattern("000,000,000.0000");
32          System.out.println(f.format(d1));
33      }
```

***Κώδικας 4.19**: Διαμόρφωση εξόδου*

Η έξοδος της outputFormat είναι η εξής:

```
9,57+9,27 = 18,84
9,56+9,28 = 18,84
0,28+0,39 = 0,66
9.856.327,56+5.627.869,29 = 15.484.196,85
9.856.327,563+5.627.869,289 = 15.484.196,852
€9.856.327,563
(€9.856.327,563)
(€128)
(€65)
009.856.327,5630
```

Η διαμόρφωση βασίζεται στην προκαθορισμένη κλάση DecimalFormat [2]. Για να τη χρησιμοποιήσετε προσθέστε στην κλάση σας την import Java.text.DecimalFormat.

Στη γραμμή 2 του κώδικα 4.19 δημιουργούμε ένα αντικείμενο της κλάσης DecimalFormat. Η μεταβλητή f είναι μεταβλητή αναφοράς. Κατά τη δημιουργία της συνδέεται με το μοτίβο (pattern), "###,###.##". Στη γραμμή 4 στέλνουμε προς εμφάνιση στην οθόνη τρεις πραγματικές τιμές, χρησιμοποιώντας τη συνάρτηση format της DecimalFormat. Η συνάρτηση format επιστρέφει μια αλφαριθμητική σειρά. Η σειρά που επιστρέφει η format υπολογίζεται με βάση την τιμή που της δίνουμε κατά την κλήση της και το μοτίβο που ορίσαμε κατά τη δημιουργία της f. Στο μοτίβο αυτό ο χαρακτήρας ',' συμβολίζει τον διαχωριστή χιλιάδων, ο χαρακτήρας '.' συμβολίζει τον διαχωριστή δεκαδικών και ο χαρακτήρας '#' συμβολίζει το ψηφίο. Επομένως, σύμφωνα με το μοτίβο, ζητάμε μια διαμόρφωση στην οποία να εμφανίζονται τα ψηφία του αριθμού κατά τρόπο που οι χιλιάδες να διαχωρίζονται μεταξύ τους, όπως επίσης και να διαχωρίζεται το δεκαδικό από το ακέραιο μέρος και το δεκαδικό μέρος να εμφανίζεται με δύο ψηφία. Η έξοδος της γραμμής 4 είναι 9.57+9.27 = 18.84. Προσέξτε καταρχάς πως έχουμε στρογγυλοποίηση του δεκαδικού μέρους. Η μεταβλητή d1 εμφανίστηκε ως 9.57 αντί του 9.566. Αντίστοιχα, η d2 εμφανίζεται ως 9.27 αντί του 9.273. Η format στρογγυλοποιεί εξ ορισμού τα δεκαδικά προς την πλησιέστερη τιμή. Το 0.57 είναι πλησιέστερο στο 0.566 σε σχέση με το 0.56. Παρόμοια, το 0.27 είναι πλησιέστερο στο 0.273 από ότι το 0.28.

Όταν όμως δεν υπάρχει πλησιέστερη τιμή, τότε η στρογγυλοποίηση γίνεται προς τον άρτιο γείτονα. Η έξοδος της γραμμής 8 είναι 9.56+9.28 = 18.84. Προσέξτε πως η τιμή του d1 από 9.565 μετατράπηκε σε 9.56. Πράγματι, η τιμή 9.565 απέχει εξίσου από το 9.56 και το 9.57. Σε αυτήν την περίπτωση η format στρογγυλοποίησε προς τον άρτιο γείτονα, δηλαδή το 6 και όχι τον περιττό, δηλαδή το 7. Αντίστοιχα, η d2 με τιμή 9.275, εμφανίζεται ως 9.28. Επομένως, η στρογγυλοποίηση έγινε προς τον άρτιο γείτονα, το 8, και όχι τον περιττό, δηλαδή το 7.

Η έξοδος της γραμμής 12, μετά τη μεταβολή των τιμών των d1 και d2 που έγινε στη γραμμή 10 και 11, είναι 0.28+0.39 = 0.66. Επομένως, το d1 με τιμή 0.275 εμφανίζεται ως 0.28 και το d2 με τιμή 0.385 εμφανίζεται ως 0.39. Εκείνο όμως που χρήζει ιδιαίτερης προσοχής είναι πως το άθροισμα 0.28+0.39 ισούται με 0.67 και όχι με 0.66. Η ασυμφωνία αυτή προκύπτει καθώς υπολογίζουμε πρώτα το άθροισμα d1+d2 και στη συνέχεια το στρογγυλοποιούμε. Επομένως, 0.275+0.385=0.610. Η στρογγυλοποίηση στα δύο δεκαδικά του 0.610 είναι προφανώς το 0.61. Είναι κατανοητό από αυτό το παράδειγμα πως η στρογγυλοποίηση κατά την εμφάνιση πραγματικών αριθμών πρέπει να γίνεται με ιδιαίτερη προσοχή.

Στη γραμμή 16 το μοτίβο "###,###.##" εφαρμόζεται σε αριθμούς με περισσότερα ψηφία από τους χαρακτήρες του χωρίς να δημιουργείται κάποιο πρόβλημα.

Στη γραμμή 18 αλλάζουμε το μοτίβο της f. Το νέο μοτίβο υποστηρίζει τρία δεκαδικά ψηφία. Έτσι η έξοδος της γραμμής 19 είναι 9,856,327.563+5,627,869.289 = 15,484,196.852.

Είναι εφικτό κατά τη διαμόρφωση των αριθμών να προσθέτουμε σταθερούς χαρακτήρες. Έτσι στη γραμμή 21 έχουμε προσθέσει το σύμβολο του Ευρώ ώστε να προηγείται της αριθμητικής τιμής. Πράγματι η έξοδος της γραμμής 22 είναι €9,856,327.563.

Στη γραμμή 23 τροποποιούμε πάλι το μοτίβο της f ώστε να περικλείεται ο αριθμός και το σύμβολο του Ευρώ από παρενθέσεις. Έτσι, η έξοδος της γραμμής 24 είναι (€9,856,327.563).

Στη γραμμή 26 επιδεικνύεται πώς το μοτίβο μπορεί να εφαρμοστεί και σε ακέραιους. Η έξοδος της γραμμής είναι (€128). Παρότι το μοτίβο προβλέπει δεκαδικά ψηφία, η format τυπώνει σωστά τους ακέραιους τύπους χωρίς δεκαδικά.

Στη γραμμή 29 εκτυπώνεται ο χαρακτήρας 'A' μέσω της format. Η έξοδος της γραμμής είναι (€65), που σημαίνει πως στη θέση του χαρακτήρα εμφανίζεται ο κωδικός του.

Σε κάποιες εφαρμογές θέλουμε να εμφανίζουμε αριθμούς σε σταθερό πλάτος συμπληρώνοντας τα στοιχεία που λείπουν με μηδενικά. Σε αυτήν την περίπτωση, προκειμένου να μην αλλοιωθεί η τιμή του αριθμού, μηδενικά μπορούν να προστεθούν μόνο στην αρχή του ακέραιου μέρους ή στο τέλος του δεκαδικού μέρους ενός αριθμού. Στη γραμμή 31 μεταβάλλουμε το μοτίβο, έτσι ώστε ο αριθμός να τοποθετείται σε πλάτος 13 ψηφίων από τα οποία τα 9 ανήκουν στο ακέραιο μέρος και τα 4 στο δεκαδικό. Η έξοδος της γραμμής 32 είναι 009,856,327.5630.

Ας σημειωθεί πως η κλάση DecimalFormat παρέχει πολλές επιπλέον δυνατότητες. Για παράδειγμα, τη δυνατότητα να καθορίσουμε ως διαχωριστικό χιλιάδων τον χαρακτήρα '.' και ως διαχωριστικό δεκαδικών τον χαρακτήρα ',' ή να αλλάξουμε τον τρόπο στρογγυλοποίησης. Περισσότερες πληροφορίες για την DecimalFormat μπορείτε να βρείτε στην τεκμηρίωση της Oracle [1].

Επιπλέον, υπάρχουν και άλλοι τρόποι για τη διαμόρφωση της εξόδου όπως η συνάρτηση format της κλάσης String [3].

4.3 Είσοδος από το πληκτρολόγιο

Για είσοδο από το πληκτρολόγιο θα χρησιμοποιούμε την προκαθορισμένη κλάση Scanner. Για να χρησιμοποιηθεί πρέπει να την εισάγετε στον κώδικά σας προσθέτοντας την import Java.util.Scanner. Με την Scanner μπορούμε να διαβάσουμε δεδομένα και από αρχεία. Ωστόσο σε αυτήν την ενότητα, περιοριζόμαστε στη χρήση της Scanner για απλή είσοδο από το πληκτρολόγιο. Η ανάγνωση αρχείων παρουσιάζεται αναλυτικά στην ενότητα 15.

Η Scanner για κάθε θεμελιώδη τύπο διαθέτει μια συνάρτηση με την οποία μπορούμε να διαβάζουμε αντίστοιχα δεδομένα. Ο πίνακας 4.6 παρουσιάζει τις συναρτήσεις που μας ενδιαφέρουν ώστε να μπορούμε να υποστηρίζουμε στα προγράμματά μας είσοδο από το πληκτρολόγιο.

Τύπος επιστροφής	**Συνάρτηση**	**Χρήση**
String	next()	Είσοδος δεδομένων τύπου String
boolean	nextBoolean()	Είσοδος δεδομένων τύπου boolean
byte	nextByte()	Είσοδος δεδομένων τύπου byte
double	nextDouble()	Είσοδος δεδομένων τύπου double

float	nextFloat()	Είσοδος δεδομένων τύπου float
int	nextInt()	Είσοδος δεδομένων τύπου int
String	nextLine()	Είσοδος δεδομένων τύπου String
long	nextLong()	Είσοδος δεδομένων τύπου long
short	nextShort()	Είσοδος δεδομένων τύπου short

Πίνακας 4.6 *Συναρτήσεις της Scanner για είσοδο δεδομένων*

Σε κάθε κλήση τους, κάθε μια από τις συναρτήσεις του πίνακα 4.6 διαβάζει μια ολοκληρωμένη σειρά δεδομένων (token). Εξ ορισμού, μια σειρά δεδομένων θεωρείται ότι τερματίζεται από την παρουσία ενός διαστήματος (whitespace). Εξαίρεση αποτελεί η συνάρτηση nextLine που διαβάζει τα δεδομένα μιας γραμμής, θεωρώντας το διάστημα ως μέρος των δεδομένων.
Στον κώδικα 4.20 επιδεικνύεται ενδεικτική χρήση των συναρτήσεων εισόδου. Στη γραμμή 2, ορίζουμε το αντικείμενο in της κλάσης Scanner μέσα από το οποίο έχουμε τη δυνατότητα να καλούμε τις συναρτήσεις εισόδου. Κατά τον ορισμό του in, το συνδέουμε με την System.in. Με αυτόν τον τρόπο καθορίζεται πως η είσοδος θα γίνεται από το πληκτρολόγιο.

```
1     static void keyboardInput() { //Κώδικας 4.20
2        Scanner in = new Scanner(System.in);
3        System.out.print("Δώστε όνομα : ");
4        String name = in.nextLine();
5
6        System.out.print("Δώστε αριθμό Μητρώου: ");
7        int aem = in.nextInt();
8
9        System.out.print("Εισόδημα έτους 2021 : ");
10       double eisodima = in.nextDouble();
11       System.out.println();
12
13       System.out.println("Όνομα: " + name);
14       System.out.println("ΑΜ : " + aem);
15       System.out.println("Εισόδημα 2021: " + eisodima);
16      // in.close();
17   }
```

Κώδικας 4.20*: Είσοδος από το πληκτρολόγιο*

Στη γραμμή 4 διαβάζουμε μια γραμμή δεδομένων. Θα μπορούσαμε να χρησιμοποιήσουμε τη συνάρτηση next(), μιας και το όνομα είναι τύπου String. Ωστόσο, αν ο χρήστης καταχωρήσει όνομα και επώνυμο διαχωρισμένα με διάστημα, η next() θα διαβάσει μόνο μέχρι το διάστημα, δηλαδή μόνο το όνομα. Ακόμη χειρότερα, η επόμενη είσοδος που επιχειρείται με την nextInt στη γραμμή 7 θα βρει ως δεδομένα εισόδου το επώνυμο του χρήστη. Θα βρει δηλαδή ένα String, ενώ περιμένει int, οπότε θα παραχθεί λάθος χρόνου εκτέλεσης.

Εφόσον διαβάσουμε σωστά το ονοματεπώνυμο του χρήστη, προχωράμε στην είσοδο του αριθμού μητρώου. Αν εδώ ο χρήστης δεν δώσει ακέραια τιμή, επίσης θα παραχθεί λάθος χρόνου εκτέλεσης. Στη συνέχεια, στη γραμμή 10, διαβάζουμε έναν αριθμό τύπου double.

Το παράδειγμα κλείνει με την in.close() σχολιασμένη. Προσοχή, μην κλείσετε ποτέ ένα αντικείμενο Scanner που είναι σχετισμένο με την System.in. Αν το κάνετε, η εφαρμογή σας δεν θα είναι σε θέση να ανοίξει ροή επικοινωνίας με το πληκτρολόγιο παρά μόνο αν επανεκκινηθεί [3].

Βέβαια, η σωστή χρήση της Scanner απαιτεί τη διαχείριση των λαθών που ενδέχεται να παραχθούν, ζήτημα με το οποίο ασχολούμαστε στην ενότητα 16.

4.4 Ασκήσεις

1. Ποια είναι η έξοδος του κώδικα που ακολουθεί;

```
int i = 0, j = 0;
System.out.print(i++ + " " + (++j) + " ");
System.out.println(i == j);
```

2. Ποια είναι η έξοδος του κώδικα που ακολουθεί; Εξηγήστε την απάντησή σας.

```
int k = 2;
System.out.println((k += 2) == (k *= 2));
```

3. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println((i++ == 0 || j++ == 0) && k++ == 0);
System.out.println(i + " " + j + " " + k);
```

4. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println(i++ == 0 || j++ == 0 && k++ == 0);
System.out.println("i++ == 0 || j++ == 0 && k++ == 0");
System.out.println(i + " " + j + " " + k);
```

5. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println(i++ == 0 && j++ == 0 || k++ == 0);
System.out.println("i++ == 0 && j++ == 0 || k++ == 0");
System.out.println(i + " " + j + " " + k);
```

6. Λάβετε από το πληκτρολόγιο 2 πραγματικούς αριθμούς και εμφανίστε στην οθόνη το γινόμενό τους. Εμφανίστε τα αποτελέσματα κατά τρόπο ώστε να προηγείται η περιγραφή τους και με 2 δεκαδικά ψηφία κατά μέγιστο.

```
Παράδειγμα
Δώσε αριθμό: 3.58
Δώσε αριθμό: 2.99
Το γινόμενο 3.58 x 2.99 = 10.7
```

7. Χρησιμοποιήστε μια μόνο println για να τυπώστε τη διεύθυνσή σας με την ακόλουθη μορφή:

```
Ονοματεπώνυμο
Οδός Αριθμός
Πόλη, ΤΚ
Χώρα
```

Υπόδειξη: Θυμηθείτε τους χαρακτήρες διαφυγής στην ενότητα 3.3.

8. Ο Γιώργος λαμβάνει μηνιαίο μισθό 1000 Ευρώ. Επίσης, λαμβάνει ενοίκιο από το διαμέρισμα στην Καλλιθέα 450 Ευρώ τον μήνα και το διπλάσιο ενοίκιο από το διαμέρισμα στου Παπάγου. Ο Γιώργος έχει ένα επιπλέον μηνιαίο εισόδημα. Τα έξοδα του μηνός Απριλίου έχουν ως εξής: Για τρόφιμα ξόδεψε τα 12/100 του συνολικού του εισοδήματος, τα καύσιμα κοστίζουν 0.2 Ευρώ το χιλιόμετρο, το Ηλεκτρικό ρεύμα 200 Ευρώ και το κόστος νερού 120 Ευρώ. Να αναπτυχθεί αλγόριθμος που ζητά από τον Γιώργο το επιπλέον μηνιαίο εισόδημα και τα χιλιόμετρα του Απριλίου και εμφανίζει κατάσταση εσόδων-εξόδων του Απριλίου σύμφωνα με το παρακάτω υπόδειγμα:

```
--------Κατάσταση Εσόδων-Εξόδων Απριλίου-----------------
Μισθός 1000
Ενοίκιο Καλλιθέας 450
Ενοίκιο Παπάγου 900
Επιπλέον μηνιαίο εισόδημα 100
-----------------------------------------------------------
Συνολικό Εισόδημα 2450

Κόστος Τροφίμων 294.00
Κόστος Καυσίμων 160.00
Κόστος Ηλεκτρικού Ρεύματος 160.00
Κόστος Νερού 120
-----------------------------------------------------------
Συνολικό Κόστος 774.00
***********************************************************
Διαφορά 1676.00
```

9. Ο Γιάννης είναι καπνιστής. Κάθε μέρα καταναλώνει ένα πακέτο τσιγάρα που κοστίζει 4 Ευρώ +20% ΦΠΑ. Επίσης, κάθε δύο μήνες χρειάζεται έναν αναπτήρα. Ρωτήστε τον χρήστη πόσο κοστίζει ένας αναπτήρας και υπολογίστε πόσο κοστίζει στον Γιάννη το κάπνισμα κάθε ημέρα και πόσο κάθε μήνα. Ποιο είναι το ετήσιο κόστος; Θεωρήστε πως όλοι οι μήνες αποτελούνται από 30 ημέρες. Για να υπολογίσετε το ημερήσιο κόστος επιμερίστε το κόστος του αναπτήρα στις ημέρες χρήσης του. Αν το ετήσιο εισόδημα του Γιάννη είναι 8.000 Ευρώ, πόσο τοις εκατό του εισοδήματός του ξοδεύει για κάπνισμα; Αν τα υπόλοιπα μηνιαία έξοδα του Γιάννη είναι 500 Ευρώ, τι ποσό από το εισόδημά του αποταμιεύει κάθε χρόνο;

Βιβλιογραφία

[1] “Operators (The Java™ Tutorials > Learning the Java Language > Language Basics).” https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html (accessed Sep. 14, 2021).

[2] “Formatting Numeric Print Output (The Java™ Tutorials > Learning the Java Language > Numbers and Strings).” https://docs.oracle.com/javase/tutorial/java/data/numberformat.html (accessed Sep. 14, 2021).

[3] K. Fogel, “If You’re Using Java’s Scanner Class for Keyboard Input, You’re Doing it Wrong - DZone Java,” DZone. https://dzone.com/articles/if-youre-using-javas-scanner-class-for-keyboard-in (accessed Sep. 14, 2021).

5 Έλεγχος Ροής

Ο έλεγχος της ροής του προγράμματος πραγματοποιείται με τις δομές επιλογής και τις επαναληπτικές δομές. Οι δομές επιλογής μας δίνουν τη δυνατότητα να δημιουργούμε πολλαπλές διαδρομές μέσα στον κώδικά μας και να επιλέγουμε κάθε φορά την καταλληλότερη ανάλογα με τις συνθήκες που επικρατούν κατά την εκτέλεση του προγράμματος. Με τις επαναληπτικές δομές ρυθμίζουμε την επανάληψη τμημάτων του κώδικά μας, επίσης, ανάλογα με συγκεκριμένες συνθήκες που επικρατούν κατά την εκτέλεση του προγράμματος. Τόσο οι δομές επιλογής όσο και οι επαναληπτικές δομές αποτελούν βασικές δομές προγραμματισμού, όχι μόνο στην Java αλλά σε κάθε προγραμματιστικό περιβάλλον.
Επιπλέον, τόσο οι δομές επιλογής όσο και οι επαναληπτικές δομές βασίζονται στον έλεγχο μιας συνθήκης (condition). Η συνθήκη είναι μία λογική έκφραση, δηλαδή μία έκφραση τύπου boolean. Για παράδειγμα, η έκφραση i>5 && j<0 είναι λογική έκφραση, καθώς λαμβάνει την τιμή true ή false ανάλογα με την τιμή των i και j.

5.1 Δομές επιλογής

Οι δομές επιλογής μας δίνουν τη δυνατότητα να καθορίζουμε διαφορετικές διαδρομές που ακολουθεί ο κώδικάς μας ανάλογα με τις συνθήκες που επικρατούν στο πρόγραμμά μας κατά τον χρόνο εκτέλεσης. Πρόκειται για θεμελιώδεις δομές που συναντώνται με τη μία ή την άλλη μορφή σε κάθε γλώσσα προγραμματισμού. Για να αντιληφθεί κανείς τη σπουδαιότητά τους αρκεί να αναλογιστεί πως χωρίς αυτές τα προγράμματα θα ήταν μια συλλογή εντολών που θα εκτελούνταν αναγκαστικά σειριακά. Με άλλα λόγια, η λήψη αποφάσεων κατά την εκτέλεση του προγράμματος θα ήταν αδύνατη. Για παράδειγμα, αν ένα πρόγραμμα διέθετε ως δεδομένα μια σειρά από ονόματα και κάθε όνομα συνδεόταν με χαρακτηρισμό φύλου, δεν θα ήταν σε θέση να προσφωνήσει τους άντρες ως κυρίους και τις γυναίκες ως κυρίες.

Η Java υποστηρίζει τις δομές επιλογής if, if…else, switch και τον τριαδικό τελεστή. Είναι όμως εφικτό να συνδυαστούν οι δομές if και if…else διαμορφώνοντας έτσι μια ιδιαίτερη δομή γνωστή ως εμφωλευμένη (nested) if. Στη συνέχεια παρουσιάζονται μια προς μια οι δομές επιλογής της Java.

5.1.1 Η επιλογή if

Η γενική μορφή της if δίνεται ακολούθως:

```
if (condition) {
     statement(s);
}
```

Η δεσμευμένη λέξη if ακολουθείται από τη συνθήκη ελέγχου μέσα σε παρενθέσεις. Αν η συνθήκη είναι αληθής, η ροή του προγράμματος μπαίνει στο μπλοκ που ακολουθεί και εκτελεί τα statements. Αν η συνθήκη είναι ψευδής, η ροή του προγράμματος μεταπηδά στην έξοδο από το μπλοκ της if.

Μέσα στο μπλοκ της if μπορούμε να έχουμε ένα ή περισσότερα statements. Αν έχουμε μόνο ένα, οι αγκύλες είναι προαιρετικές, ωστόσο, συνηθίζεται να τοποθετούνται.

```
static void simpleIf() { //Κώδικας 5.1
    int i = 1, j = -1;
    if (i > 0 && j < 1) {
        System.out.println("i μεγαλύτερο από το 0");
        System.out.println("j μικρότερο από το 1");
    }
    System.out.println("Μετά τον έλεγχο της έκφρασης \"i>0 && j<1\"");
}
```

Κώδικας 5.1 *Παράδειγμα απλής if*